

Slider de Soluciones

Tutorial para construir `SoluctionSection` desde cero, por partes: primero el HTML de cada bloque y después sus estilos, y al final la funcionalidad del carrusel. El resultado es el diseño que ya tienes, con el slider funcionando card por card como en `MyServicesSection`. Las animaciones con GSAP no entran aquí.

Cómo usar el tutorial

Cada parte tiene dos pasos. Primero escribes el HTML `HTML` sin clases y compruebas en el navegador que el contenido aparece. Después agregas los estilos `CSS` y lees por qué va cada clase. La parte 3 `JS` agrega la lógica sin tocar el diseño.

Así quedan las capas de la sección. El panel lateral es `absolute` (decorativo, detrás) y el contenedor del carrusel va en el flujo normal (adelante):

```
section#mySoluctionSection – relative, overflow-x-clip
```

La sección completa. Ancla del panel y recorte horizontal.

Parte 1 · panel lateral – absolute, z-0

Fondo decorativo con el título, el brillo y la cuadrícula.

Parte 2 · contenedor dots + carrusel – relative, z-10

Va en el flujo y le da la altura a la sección.

dots

Un punto por posición del carrusel.

wrapper → clip → carrusel (carouselRef) → track (ul)

El track es lo único que se mueve. Dentro, un li por card.

Antes de empezar, verifica que tienes `src/data/solutions.js` (ya lo tienes, con 4 soluciones) y estos imports:

SoluctionSection.jsx – imports (parte 1 y 2)

```
//datos de las cards
import { solutions } from '../..data/solutions';

//iconos
import { ArrowUpRight, Clock } from 'lucide-react'
```

Los imports de React (`useState` , `useEffect` , `useRef`) y de `useNavigate` se agregan en la parte 3, cuando de verdad se usan.

Parte 1

Panel lateral decorativo

El bloque gris redondeado de la izquierda con el título “SOLUCIONES QUE CONSTRUYO”, un brillo en la esquina y una cuadrícula inclinada.

1.1 HTML HTML

Estructura del panel

El componente empieza con la `<section>`. Dentro va el panel, que tiene tres hijos: el brillo, el bloque de texto y la cuadrícula SVG.

parte 1.1 – sin estilos

```
const SolutionSection = () => {
  return (
    <section id="mySolutionSection">
      {/* Panel lateral decorativo – entra desde la izquierda */}
      <div>
        {/* Brillo decorativo – esquina superior derecha */}
        <div></div>

        {/* Bloque de texto */}
        <div>
          <div>
            <h2>SOLUCIONES QUE CONSTRUYO</h2>
            <p>
              Software, datos, automatización e IA aplicados a
              problemas reales de negocio. Cada capacidad está respaldada por proyectos.
            </p>
          </div>
        </div>

        {/* Cuadrícula decorativa */}
        <svg aria-hidden="true">
          <defs>
            <pattern id="grid_pattern_right" width="45" height="38"
              patternUnits="userSpaceOnUse">
              <path d="M.5 38V.5H48" fill="none" strokeWidth="1" />
            </pattern>
          </defs>
        </svg>
      </div>
    </section>
  )
}
```

```

    <rect width="100%" height="100%" fill="url(#grid_pattern_right)"
  />

    <g opacity="0.4">
      <rect width="44" height="39" x="92" y="38"
fill="currentColor" />
      <rect width="44" height="39" x="182" y="114"
fill="currentColor" />
      <rect width="44" height="39" x="135" y="190"
fill="currentColor" />
    </g>
  </svg>
</div>

  {/* Contenedor dots + carrusel - parte 2 */}
</section>
)
}

export default SolutionSection

```

- El `<pattern>` dibuja una celda de 45×38px con dos líneas (izquierda y arriba). El `<rect>` de 100% la repite en todo el SVG y así se forma la cuadrícula.
- Los tres `<rect>` del `<g>` son celdas “rellenas” sueltas que le dan textura. `fill="currentColor"` toma el color del texto, que se define con una clase en el paso de estilos.
- `aria-hidden="true"` le indica a los lectores de pantalla que el SVG es solo decoración.

En el navegador verás el título, el párrafo y la cuadrícula del SVG en su tamaño por defecto (300×150px). El brillo todavía no se ve porque está vacío.

1.2 Estilos `CSS`

Estilos del panel

ELEMENTO	CLASES / STYLE	PARA QUÉ
<code>section</code>	<code>relative w-full mb-10</code> <code>overflow-x-clip</code> + <code>style={{</code> <code>scrollMarginTop: "24px" }}</code>	<code>relative</code> hace que el panel <code>absolute</code> se ubique respecto a la sección. <code>overflow-x-clip</code> recorta las cards que quedan a la derecha sin crear scroll interno (con <code>overflow-x-hidden</code> el eje vertical pasaría a <code>auto</code>). <code>scrollMarginTop</code> deja 24px de aire cuando el navbar salta a <code>#mySolutionSection</code> .

ELEMENTO	CLASES / STYLE	PARA QUÉ
Panel: posición	<code>absolute top-20 bottom-0 left-0 z-0</code>	Empieza 80px abajo del borde de la sección y llega hasta el final. Así se adapta a la altura del contenido en cualquier pantalla. <code>z-0</code> lo deja detrás del carrusel.
Panel: ancho	<code>w-[85%] sm:w-[80%] lg:w-1/2</code>	Ocupa casi todo en móvil y la mitad en escritorio.
Panel: apariencia	<code>overflow-hidden bg-gray-50 dark:bg-zinc-900 border border-zinc-200/50 dark:border-zinc-800/50 rounded-tr-[2rem] rounded-br-[2rem] sm:rounded-tr-[3rem] sm:rounded-br-[3rem] + sombras shadow-[...] / dark:shadow-[...]</code>	Solo el lado derecho va redondeado, porque el izquierdo toca el borde de la pantalla. <code>overflow-hidden</code> recorta el brillo y la cuadrícula dentro de esas esquinas.
Panel: rendimiento	<code>will-change-transform</code>	Prepara la capa para la animación GSAP de la fase siguiente. Por ahora no hace nada visible y puedes dejarlo o agregarlo después.
Brillo	<code>absolute top-0 right-0 w-[200px] h-[400px] bg-gradient-to-r from-zinc-600 to-zinc-400 opacity-40 dark:from-[#fff]/30 dark:to-[#e9e9e9]/30 dark:opacity-100 pointer-events-none + maskImage: radial-gradient(farthest-side at right, white, transparent)</code>	Un rectángulo gris pegado a la esquina. La máscara radial lo deja visible solo cerca del borde derecho, y se desvanece hacia adentro como un halo. <code>pointer-events-none</code> evita que tape clics.
Bloque de texto	<code>w-[85%] sm:w-[90%] ml-6 sm:ml-8 md:ml-10 mr-4 sm:mr-6 md:mr-10 my-6 sm:my-8 md:my-10 relative z-10 pb-3 border-b border-zinc-300 dark:border-zinc-700</code>	Márgenes que crecen con la pantalla. <code>relative z-10</code> lo pone encima de la cuadrícula. <code>border-b</code> es la línea bajo la descripción.
div interno	<code>md:items-start md:justify-between md:gap-6</code>	Viene del original. Como el div no es <code>flex</code> , estas clases no tienen efecto; puedes dejarlas o quitarlas.

ELEMENTO	CLASES / STYLE	PARA QUÉ
h2	text-2xl sm:text-3xl md:text-4xl lg:text-5xl font-bold text-zinc-700 dark:text-zinc-100	El título crece por breakpoint, de 24px a 48px.
p	max-w-md text-xs sm:text-sm text-zinc-600 dark:text-zinc-300 text-justify mt-2 sm:mt-3	max-w-md (448px) evita líneas muy largas en escritorio.
svg	absolute bottom-0 right-0 h-[70%] w-full fill-black/[0.02] stroke-black/[0.05] dark:fill-white/[0.01] dark:stroke-white/[0.03] pointer-events-none + transform: skewY(-12deg), transformOrigin: bottom right	Ocupa el 70% inferior del panel. skewY inclina la cuadrícula para darle perspectiva. Las líneas son casi transparentes, apenas una textura.
rect del g	text-zinc-400/20 dark:text-zinc-600/20	Define elcurrentColor que usan como relleno.

parte 1.2 – panel con estilos

```
const SolutionSection = () => {
  return (
    <section id="mySolutionSection" className='relative w-full mb-10 overflow-x-clip' style={{ scrollMarginTop: "24px" }}>
      {/* Panel lateral decorativo – entra desde la izquierda */}
      <div className='overflow-hidden absolute top-20 bottom-0 left-0 w-[85%] sm:w-[80%] lg:w-1/2 bg-gray-50 dark:bg-zinc-900 shadow-[0_2px_8px_0_rgba(99,99,99,0.2)] dark:shadow-[0_8px_24px_-4px_rgba(0,0,0,0.5),0_32px_64px_-8px_rgba(0,0,0,0.35)] border border-zinc-200/50 dark:border-zinc-800/50 z-0 rounded-tr-[2rem] rounded-br-[2rem] sm:rounded-tr-[3rem] sm:rounded-br-[3rem] will-change-transform'>
        {/* Brillo decorativo – esquina superior derecha */}
        <div className='absolute top-0 right-0 w-[200px] h-[400px] bg-gradient-to-r from-zinc-600 to-zinc-400 opacity-40 dark:from-[#fff]/30 dark:to-[#e9e9e9]/30 dark:opacity-100 pointer-events-none' style={{ maskImage: "radial-gradient(farthest-side at right, white, transparent)", WebkitMaskImage: "radial-gradient(farthest-side at right, white, transparent)" }}></div>

        {/* Bloque de texto */}
        <div className='w-[85%] sm:w-[90%] ml-6 sm:ml-8 md:ml-10 mr-4 sm:mr-6 md:mr-10 my-6 sm:my-8 md:my-10 relative z-10 pb-3 border-b border-zinc-300'>

```

```

dark:border-zinc-700'>
    <div className='md:items-start md:justify-between md:gap-6'>
        <h2 className='text-2xl sm:text-3xl md:text-4xl lg:text-5xl
font-bold text-zinc-700 dark:text-zinc-100'>
            SOLUCIONES QUE CONSTRUYO
        </h2>
        <p className='max-w-md text-xs sm:text-sm text-zinc-600
dark:text-zinc-300 text-justify mt-2 sm:mt-3'>
            Software, datos, automatización e IA aplicados a
            problemas reales de negocio. Cada capacidad está respaldada por proyectos.
        </p>
    </div>
</div>

    {/* Cuadrícula decorativa */}
    <svg aria-hidden="true" className="absolute bottom-0 right-0 h-[70%]
w-full fill-black/[0.02] stroke-black/[0.05] dark:fill-white/[0.01] dark:stroke-
white/[0.03] pointer-events-none" style={{ transform: "skewY(-12deg)",
transformOrigin: "bottom right" }}>
        <defs>
            <pattern id="grid_pattern_right" width="45" height="38"
patternUnits="userSpaceOnUse">
                <path d="M.5 38V.5H48" fill="none" strokeWidth="1" />
            </pattern>
        </defs>
        <rect width="100%" height="100%" fill="url(#grid_pattern_right)"
/>

        <g opacity="0.4">
            <rect width="44" height="39" x="92" y="38"
fill="currentColor" className="text-zinc-400/20 dark:text-zinc-600/20" />
            <rect width="44" height="39" x="182" y="114"
fill="currentColor" className="text-zinc-400/20 dark:text-zinc-600/20" />
            <rect width="44" height="39" x="135" y="190"
fill="currentColor" className="text-zinc-400/20 dark:text-zinc-600/20" />
        </g>
    </svg>
</div>

    {/* Contenedor dots + carrusel - parte 2 */}
</section>
)
}

```

Todavía no se ve el panel. La sección solo tiene hijos `absolute`, así que mide 0px de alto. Es el mismo problema que vimos al principio. Se resuelve solo en la parte 2, cuando el contenedor del carrusel le da altura. Para revisar el panel mientras tanto, puedes poner un `min-h-[700px]` temporal en la sección.

Parte 2

Contenedor dots + carrusel

Los dots con el diseño original, las capas del carrusel y la card de cada solución con su botón “Ver proyectos”. Todo estático: se ve como el resultado final, pero todavía no se mueve.

2.1 HTML

HTML

Estructura del contenedor y de la card

Va dentro de la `<section>`, después del panel. Los dots son dos botones fijos por ahora (uno activo y uno inactivo); en la parte 3 se generan solos.

parte 2.1 – contenedor, sin estilos

```
{/* Contenedor dots + carrusel */}
<div>
  {/* Dots – por ahora fijos: el 1 activo, el 2 inactivo */}
  <div>
    <button type="button" aria-label="Ir al slide 1" />
    <button type="button" aria-label="Ir al slide 2" />
  </div>

  {/* Wrapper – nunca se transforma */}
  <div>
    {/* Clip – recorta a los lados sin cortar sombras */}
    <div>
      {/* Carrusel – aquí irá carouselRef en la parte 3 */}
      <div role="region" aria-roledescription="carrusel" aria-label="Soluciones">
        {/* Track – lo único que se mueve */}
        <ul>
          {solutions.map((solution) => (
            <li key={solution.id}>
              <SolutionCard solution={solution} />
            </li>
          ))}
        </ul>
      </div>
    </div>
  </div>
```

```
</div>
</div>
```

Y la card, como una función aparte encima de `SolutionSection` :

parte 2.1 – SolutionCard, sin estilos

```
//funcion con diseño de las cards
function SolutionCard({ solution }) {
  return (
    <article>
      {/* Fondo de puntos */}
      <div aria-hidden="true" />

      {/* Área visual: imagen si existe, si no el placeholder */}
      <figure>
        {solution.image ? (
          <img src={solution.image} alt={solution.alt} loading="lazy"
draggable={false} />
        ) : (
          <div>
            <svg width="28" height="28" viewBox="0 0 28 28" fill="none"
aria-hidden="true">
              <path d="M14 3v10m0 0 4-4m-4 4-4-4" stroke="currentColor"
strokeWidth="1.6" strokeLinecap="round" strokeLinejoin="round" />
              <path d="M4 18v4a2 2 0 0 0 2 2h16a2 2 0 0 0 2-2v-4"
stroke="currentColor" strokeWidth="1.6" strokeLinecap="round" />
            </svg>
            <span>Proyectos en camino</span>
          </div>
        )}
        {/* Degradado inferior que funde la imagen con la card */}
      <div />
    </figure>

    {/* Contenido textual */}
    <div>
      <h3>{solution.title}</h3>
      <hr />
      <p>{solution.description}</p>

      {/* CTA – habilitado solo si la solución está activa */}
      <div>
        {solution.active ? (
          <button type="button">
            Ver proyectos
            <ArrowUpRight />
          </button>
        )}
      </div>
    </div>
  )
}
```

```

    ) : (
      <button type="button" disabled title="Próximamente">
        <Clock />
        Ver proyectos
      </button>
    )}
  </div>
</div>
</article>
)
}

```

- **¿Por qué tantas capas?** Cada una tiene un trabajo. El *wrapper* no se mueve nunca. El *clip* recorta a los lados. El *carrusel* se mide y recibe el arrastre. El *track* (`ul`) es lo que se desliza. Separarlas evita que un estilo de una afecte el cálculo de otra.
- `solution.image ? ... : ...` elige entre la imagen y el placeholder. En tu `solutions.js` las imágenes son `""`; un string vacío cuenta como falso, así que hoy se ve el placeholder.
- `draggable={false}` evita que el navegador arrastre la imagen como archivo cuando en la parte 3 arrastres el carrusel.
- **El botón deshabilitado** lleva `disabled` y `title="Próximamente"`, que aparece como tooltip al pasar el mouse.

En el navegador verás las 4 cards una debajo de otra, sin estilos. La sección ya tiene altura y el panel debería aparecer de fondo.

2.2 Estilos CSS

Estilos del contenedor, los dots y la card

Las clases largas que se repiten se guardan en constantes arriba del archivo. Así el JSX queda legible y la parte 3 puede reutilizarlas para cambiar el dot activo.

Constantes de estilo

parte 2.2 – constantes (arriba del archivo, después de los imports)

```

// ===== ESTILOS =====

//Card base – `relative` para el fondo de puntos.
//Alto: la sección nunca pasa del 95% de la ventana. Se resta lo fijo
//(pt + dots + pb: 23.5rem en móvil, 26rem desde md) y se limita entre 260px y 330px

```

```

const CARD_BASE =
  'relative flex h-[clamp(260px,calc(95svh_-_23.5rem),330px)] md:h-[clamp(260px,calc(95svh_-_26rem),330px)] flex-col overflow-hidden rounded-[20px] border border-black/5 bg-gray-100 ' +
  'shadow-[0_2px_12px_0_rgba(0,0,0,0.08),0_1px_3px_0_rgba(0,0,0,0.05)] dark:border-white/10 dark:bg-zinc-800'

//Hover solo en cards activas (sin cursor-pointer: la acción es el botón)
const CARD_ACTIVE =
  'transition duration-500 hover:-translate-y-1 ' +
  'hover:shadow-[0_12px_32px_0_rgba(0,0,0,0.13),0_2px_8px_0_rgba(0,0,0,0.08)]'

//Fondo de puntos – reemplaza a DotPatternCanvas con CSS puro
const DOT_PATTERN = {
  backgroundImage: 'radial-gradient(circle, rgba(128,128,128,0.13) 1.1px, transparent 1.2px)',
  backgroundSize: '18px 18px',
  maskImage: 'linear-gradient(to bottom, black, transparent)',
  WebkitMaskImage: 'linear-gradient(to bottom, black, transparent)',
}

//Botón "Ver proyectos" – mismos estilos que los CTAs de MyServicesSection
const BTN_BASE =
  'inline-flex w-full items-center justify-center gap-1.5 rounded-lg px-3 py-2 text-[11.5px] font-semibold'

const BTN_ENABLED =
  'bg-zinc-800 text-white hover:bg-blue-600 transition-colors duration-200 ' +
  'dark:bg-zinc-100 dark:text-zinc-900 dark:hover:bg-blue-500 dark:hover:text-white'

const BTN_DISABLED =
  'cursor-not-allowed bg-zinc-200/70 text-zinc-400 dark:bg-zinc-700/50 dark:text-zinc-500'

//Ancho de cada slide – 1 / 2 / 3 cards (gap-4 = 1rem entre cards)
const SLIDE_WIDTH = 'w-full md:w-[calc((100%_-_1rem)/2)] lg:w-[calc((100%_-_2rem)/3)]'

//Dots – diseño original de MyServicesSection
const DOT_BASE = 'h-2 rounded-full transition-all duration-300'
const DOT_ACTIVE = 'w-8 bg-zinc-600 dark:bg-zinc-300'
const DOT_INACTIVE = 'w-2 bg-zinc-300 hover:bg-zinc-400 dark:bg-zinc-600 dark:hover:bg-zinc-500'

```

Contenedor y dots

ELEMENTO	CLASES / STYLE	PARA QUÉ
Contenedor	<code>relative z-10 px-6 sm:px-8 md:px-10 pt-70 md:pt-80 pb-12</code>	<code>relative z-10</code> lo pinta encima del panel: sin posición, un bloque normal queda <i>debajo</i> de un hermano <code>absolute . pt-70 / md:pt-80</code> (280 / 320px) bajan los dots y las cards por debajo del título del panel.
Fila de dots	<code>flex items-center gap-2 mt-4 mb-6</code>	Diseño original: 8px entre dots, 16px arriba y 24px antes de las cards.
Dot	<code>DOT_BASE + DOT_ACTIVE</code> o <code>DOT_INACTIVE</code>	8px de alto y redondo. El activo se estira a 32px y es más oscuro. <code>transition-all duration-300</code> hace que el cambio de ancho sea suave.
Wrapper	<code>relative w-full</code>	Capa fija. En la fase GSAP aquí irá el trigger de la animación de las cards.
Clip	<code>style={{ clipPath: "inset(-16px -12px -16px -12px)", padding: "16px 12px", margin: "-16px -12px" }}</code>	El padding da espacio para la sombra y la elevación del hover, y el margen negativo lo compensa para que no se mueva nada. <code>clipPath</code> recorta a 12px por cada lado: como el gap entre cards mide 16px, la card siguiente no asoma.
Carrusel	<code>w-full select-none</code>	Sin padding, para que en la parte 3 su ancho sea exactamente el del track. <code>select-none</code> evita seleccionar texto al arrastrar.
Track (<code>ul</code>)	<code>flex gap-4</code>	Las cards en fila con 16px entre ellas.
Slide (<code>li</code>)	<code>shrink-0 + SLIDE_WIDTH</code>	Sin un ancho, cada <code>li</code> mide lo que su texto en una línea (fue el error de la card enorme). <code>shrink-0</code> impide que flex las encoja para que quepan todas.

Alto de la card: 95% de la ventana como máximo

Lo único que se adapta al alto de la ventana es la card. El resto del contenedor es fijo:

PIEZA	MÓVIL	DESDE MD
Espacio superior (<code>pt-70 / md:pt-80</code>)	280	320
Dots (<code>mt-4 + h-2 + mb-6</code>)	48	48

PIEZA	MÓVIL	DESDE MD
Espacio inferior (<code>pb-12</code>)	48	48
Fijo	376 = 23.5rem	416 = 26rem

Por eso la card mide `clamp(260px, 95svh - fijo, 330px)` : nunca más de 330px, nunca menos de 260px (lo mínimo para que quepan el texto y el botón), y en el medio lo que deje la ventana. Se usa `svh` y no `vh` porque en el celular no cambia cuando la barra del navegador aparece o se esconde.

Tu archivo actual resta 22rem y 24.5rem porque los dots tenían solo `mb-4` . Con el diseño original (`mt-4 mb-6`) los dots ocupan 24px más, y por eso aquí se resta 23.5rem y 26rem.

Card

ELEMENTO	CLASES / STYLE	PARA QUÉ
<code>article</code>	<code>CARD_BASE</code> + <code>CARD_ACTIVE</code> si <code>solution.active</code>	Columna flex con el alto calculado. Solo las cards activas se elevan 4px al pasar el mouse.
Fondo de puntos	<code>pointer-events-none absolute inset-0</code> + <code>style={DOT_PATTERN}</code>	Un <code>radial-gradient</code> repetido cada 18px dibuja los puntos, y la máscara lineal los desvanece hacia abajo. Hace lo mismo que el canvas del original, pero sin JavaScript.
<code>figure</code>	<code>relative z-[1] min-h-20 flex-1 overflow-hidden</code>	<code>flex-1</code> : la imagen toma el espacio que sobre. <code>min-h-20</code> : nunca baja de 80px. Así, cuando la card se achica, solo se achica la imagen.
<code>img</code>	<code>pointer-events-none h-full w-full object-cover</code>	<code>object-cover</code> recorta la imagen sin deformarla.
Placeholder	<code>flex h-full w-full flex-col items-center justify-center gap-3 bg-zinc-400/20 dark:bg-zinc-950/40</code>	Centra el ícono y el texto.
Ícono del placeholder	<code>text-zinc-500 opacity-30</code> <code>dark:text-zinc-400</code>	Le da color al <code>currentColor</code> de los trazos.

ELEMENTO	CLASES / STYLE	PARA QUÉ
Texto del placeholder	<code>text-[10px] uppercase</code> <code>tracking-[0.5px] text-zinc-400</code> <code>dark:text-zinc-500</code>	
Degradado	<code>pointer-events-none absolute</code> <code>inset-x-0 bottom-0 h-16 bg-</code> <code>linear-to-b from-transparent</code> <code>to-gray-100 dark:to-zinc-800</code>	Termina en el mismo color de fondo de la card, así la imagen se funde con el texto.
Contenido	<code>relative z-[2] flex shrink-0</code> <code>flex-col px-5 pt-3 pb-4</code>	<code>shrink-0</code> : el texto y el botón siempre conservan su tamaño. <code>z-[2]</code> lo pone encima de los puntos.
<code>h3</code>	<code>mb-2 text-</code> <code>[clamp(1.05rem,1.6vw,1.25rem)]</code> <code>leading-tight font-bold</code> <code>tracking-tight text-zinc-800</code> <code>dark:text-zinc-50</code>	Crece con el ancho de la ventana, entre 16.8px y 20px, como en el original.
<code>hr</code>	<code>mb-3 border-zinc-200</code> <code>dark:border-white/10</code>	La línea divisoria.
<code>p</code>	<code>line-clamp-3 text-[12.5px]</code> <code>leading-[1.65] text-zinc-500</code> <code>dark:text-zinc-400</code>	Máximo 3 líneas, con “...” al final.
Contenedor del botón	<code>mt-auto pt-3</code>	<code>mt-auto</code> empuja el botón al fondo.
Botón	<code>BTN_BASE + BTN_ENABLED</code> o <code>BTN_DISABLED</code> ; íconos <code>h-3 w-3</code>	Habilitado: oscuro, azul al hover, flecha a la derecha. Deshabilitado: gris, reloj a la izquierda, <code>cursor-not-allowed</code> .

parte 2.2 – SolutionCard con estilos

```
//funcion con diseño de las cards
function SolutionCard({ solution }) {
  return (
    <article className={`${CARD_BASE} ${solution.active ? CARD_ACTIVE : ''}`}>
      {/* Fondo de puntos */}
      <div aria-hidden="true" className="pointer-events-none absolute inset-0"
style={DOT_PATTERN} />

      {/* Área visual */}
      <figure className="relative z-[1] min-h-20 flex-1 overflow-hidden">
```



```

    })
  </div>
</div>
</article>
)
}

```

parte 2.2 – contenedor con estilos (dentro de la section, después del panel)

```

{/* Contenedor dots + carrusel */}
<div className="relative z-10 px-6 pt-70 md:pt-80 pb-12 sm:px-8 md:px-10">

  {/* Dots – diseño original, por ahora fijos */}
  <div className="mt-4 mb-6 flex items-center gap-2">
    <button type="button" aria-label="Ir al slide 1" className={` ${DOT_BASE}
    ${DOT_ACTIVE}`} />
    <button type="button" aria-label="Ir al slide 2" className={` ${DOT_BASE}
    ${DOT_INACTIVE}`} />
  </div>

  {/* Wrapper – nunca se transforma */}
  <div className="relative w-full">
    {/* Clip – recorta a los lados sin cortar sombras */}
    <div style={{ clipPath: "inset(-16px -12px -16px -12px)", padding: "16px
    12px", margin: "-16px -12px" }}>
      {/* Carrusel – aquí irá carouselRef en la parte 3 */}
      <div role="region" aria-roledescription="carrusel" aria-
      label="Soluciones" className="w-full select-none">
        {/* Track */}
        <ul className="flex gap-4">
          {solutions.map((solution) => (
            <li key={solution.id} className={`shrink-0 ${SLIDE_WIDTH}`}>
              <SolutionCard solution={solution} />
            </li>
          ))}
        </ul>
      </div>
    </div>
  </div>
</div>

```

Revisa con DevTools (Ctrl+Shift+M): 3 cards desde 1024px, 2 entre 768 y 1023px y 1 por debajo. La cuarta card queda oculta a la derecha, esperando a que el carrusel se mueva.

Parte 3

Funcionalidad del carrusel

El carrusel avanza card por card con los dots, el arrastre del mouse y el swipe del dedo. El botón “Ver proyectos” navega a la página de cada solución. No se toca ninguna clase de la parte 2.

La idea, en cinco piezas:

1 slidesPerView:

cuántas cards caben (1, 2 o 3).

2 maxIndex =

cards – slidesPerView: la última posición válida.

3 currentIndex: la

primera card visible.

4 translateX: los px

que llevas arrastrados.

5 El track se corre

$\text{índice} \times (\text{card} + \text{gap})$
+ arrastre.

3.1

Base

JS

Imports, estado y refs

3.1 – imports (arriba del archivo)

```
import { useState, useEffect, useRef } from "react";
import { useNavigate } from "react-router-dom";
```

3.1 – constante (junto a las de estilos)

```
const GAP = 16; // px – igual a gap-4 del track
```

3.1 – al inicio de SolutionSection

```
const navigate = useNavigate();
```

```
// — Estado del carrusel —
const [currentIndex, setCurrentIndex] = useState(0); // primera card visible
const [slidesPerView, setSlidesPerView] = useState(3); // 3 / 2 / 1 según el ancho
const [isDragging, setIsDragging] = useState(false); // arrastre con mouse en curso
const [startX, setStartX] = useState(0); // X donde empezó el arrastre
const [translateX, setTranslateX] = useState(0); // px arrastrados (-200 a 200)
const [startTime, setStartTime] = useState(0); // para medir la velocidad

// — Refs del carrusel (solo los necesarios) —
const carouselRef = useRef(null); // elemento: se mide y recibe el touch
const isDraggingRef = useRef(false); // valor: arrastre en curso, sin esperar al render
const currentIndexRef = useRef(0); // valor: índice actual para el listener táctil
const maxIndexRef = useRef(0); // valor: índice máximo para el listener táctil
```

Los 4 refs que sí van, y por qué

REF	DÓNDE VA	POR QUÉ ES NECESARIO
<code>carouselRef</code>	<code>ref=</code> en el <code>div</code> con <code>role="region"</code>	1) Medir su <code>offsetWidth</code> para saber cuántos px mover el track. 2) Registrar el listener táctil nativo (paso 3.8). Va en esa capa porque no tiene padding.
<code>isDraggingRef</code>	Ningún elemento	<code>mousemove</code> se dispara muchas veces antes de que React vuelva a renderizar. El estado <code>isDragging</code> tendría el valor viejo; el ref lo tiene al instante.
<code>currentIndexRef</code>	Ningún elemento	El listener táctil se registra una sola vez al montar. Si leyera <code>currentIndex</code> , vería siempre 0.
<code>maxIndexRef</code>	Ningún elemento	Lo mismo, para el índice máximo.

Refs que no van aquí: `sectionRef`, `panelRef`, `panelGlowRef`, `panelTitleRef`, `panelDescRef`, `dotsRef` y `carouselWrapperRef`. En `MyServicesSection` solo los usa GSAP, así que se agregan en la fase de animaciones.

DATO	TIPO	REGLA
<code>currentIndex</code> , <code>translateX</code> , <code>isDragging</code> , <code>slidesPerView</code>	estado	Cambian lo que se ve (posición, dot activo, cursor). Cada cambio tiene que re-renderizar.

DATO	TIPO	REGLA
<code>startX</code> , <code>startTime</code>	estado	Así está en <code>MyServicesSection</code> . Solo los leen los handlers.

3.2 Responsive JS

Cuántas cards caben

Usa los mismos cortes que `SLIDE_WIDTH` (768 y 1024px), así el JavaScript y el CSS coinciden en cuántas cards se ven.

3.2

```
// — Cards por vista según el ancho de la ventana —
useEffect(() => {
  const handleResize = () => {
    if (window.innerWidth ≥ 1024) setSlidesPerView(3);
    else if (window.innerWidth ≥ 768) setSlidesPerView(2);
    else setSlidesPerView(1);
  };
  handleResize();
  window.addEventListener("resize", handleResize);
  return () => window.removeEventListener("resize", handleResize);
}, []);
```

El ancho de las cards lo siguen dando las clases de `SLIDE_WIDTH`, así que no hace falta el `style={{ width }}` de `MyServicesSection`. Si cambias un breakpoint, cámbialo en los dos lugares.

3.3 Límites JS

Índice máximo

Como avanza card por card, la primera card visible va de 0 a `solutions.length - slidesPerView`:

PANTALLA	CARDS POR VISTA	MAXINDEX	DOTS
≥ 1024px	3	1	2

PANTALLA	CARDS POR VISTA	MAXINDEX	DOTS
768–1023px	2	2	3
< 768px	1	3	4

3.3

```
const maxIndex = Math.max(0, solutions.length - slidesPerView);

// copias para el listener táctil – se actualizan en cada render
currentIndexRef.current = currentIndex;
maxIndexRef.current = maxIndex;

// Si al agrandar la ventana el índice queda fuera de rango, se corrige.
// Ej.: en móvil estás en la card 4 (índice 3) y pasas a escritorio, donde el máximo es 1.
useEffect(() => {
  const maxPossible = Math.max(0, solutions.length - slidesPerView);
  if (currentIndex > maxPossible) setCurrentIndex(maxPossible);
}, [slidesPerView, currentIndex]);
```

3.4 Navegación

JS

Siguiente y anterior

Avanzan o retroceden una card. En un extremo dan la vuelta, pero el arrastre solo las llama cuando no estás en un extremo, así que en la práctica el carrusel se detiene en la primera y en la última.

3.4

```
const nextSlide = () => setCurrentIndex(p => p ≥ maxIndex ? 0 : p + 1);
const prevSlide = () => setCurrentIndex(p => p ≤ 0 ? maxIndex : p - 1);
```

3.5 Dots

JS

Dots dinámicos

Reemplaza los dos botones fijos de la parte 2 por uno por cada posición. Las constantes de estilo no cambian; solo se decide cuál recibe `DOT_ACTIVE`.

3.5 – reemplaza el bloque de dots

```
{/* Dots – uno por posición posible */}
{maxIndex > 0 && (
  <div className="mt-4 mb-6 flex items-center gap-2">
    {Array.from({ length: maxIndex + 1 }).map((_, i) => (
      <button
        key={i}
        type="button"
        onClick={() => setCurrentIndex(i)}
        aria-label={`Ir al slide ${i + 1}`}
        className={` ${DOT_BASE} ${i === currentIndex ? DOT_ACTIVE :
DOT_INACTIVE}`}
      />
    ))}
  </div>
)}
```

Si todas las cards cupieran en pantalla (`maxIndex === 0`), los dots desaparecerían y la sección mediría 48px menos. Con 4 soluciones eso no pasa: como máximo caben 3.

3.6 Desplazamiento JS

Mover el track

Si el carrusel mide `W` y caben `n` cards, cada card mide $(W - (n - 1) \cdot 16) / n$. Un paso es una card más el gap. El track se corre `currentIndex` pasos a la izquierda, más lo arrastrado.

Por ejemplo, a 1280px el contenedor (`md:px-10`) deja $W = 1200$ px. Cada card mide $(1200 - 32) / 3 \approx 389$ px y un paso es 405px, así que en el índice 1 el track queda en `translateX(-405px)`.

3.6 – función

```
const getTransformValue = () => {
  if (!carouselRef.current) return "translateX(0px)";
  const containerWidth = carouselRef.current.offsetWidth;
  const cardWidth = (containerWidth - (slidesPerView - 1) * GAP) / slidesPerView;
  const offset = currentIndex * (cardWidth + GAP);
  return `translateX(${ -offset + translateX}px)`;
};
```

3.6 – en el track (ul)

```
<ul
  className="flex gap-4"
  style={{
    transform: getTransformValue(),
    transition: isDragging ? "none" : "transform 300ms ease-out",
  }}
>
```

Mientras arrastras con el mouse no hay transición, así el track sigue al cursor sin retraso. Al soltar, los 300ms lo acomodan en la card que corresponde. Es la respuesta del slider al gesto, no una animación GSAP.

3.7 Mouse JS

Arrastre con mouse

1. **mousedown:** si el clic cae sobre un botón (un dot o “Ver proyectos”), no arranca el arrastre y el botón funciona normal. Si no, guarda la X y la hora.
2. **mousemove:** el track sigue al mouse, con un límite de ± 200 px.
3. **mouseup:** calcula la velocidad. Un gesto rápido (más de 0.5 px/ms) cambia de card con 50px; uno lento necesita 100px. Si no llega, vuelve a su lugar.
4. **mouseleave:** si sales del carrusel mientras arrastras, se cierra el gesto.

3.7 – handlers

```
// — Arrastre con mouse —
const handleMouseDown = (e) => {
  if (e.target.tagName === "BUTTON" || e.target.closest("button")) return;
  setIsDragging(true);
  isDraggingRef.current = true;
  setStartX(e.pageX);
  setStartTime(Date.now());
  setTranslateX(0);
  e.preventDefault(); // evita seleccionar texto o arrastrar imágenes
};

const handleMouseMove = (e) => {
  if (!isDraggingRef.current) return;
  e.preventDefault();
  setTranslateX(Math.max(-200, Math.min(200, e.pageX - startX)));
};
```

```

const handleMouseUp = () => {
  if (!isDraggingRef.current) return;
  setIsDragging(false);
  isDraggingRef.current = false;
  const elapsed = Date.now() - startTime;
  const velocity = Math.abs(translateX) / elapsed;
  if (Math.abs(translateX) < 5 && elapsed < 200) { setTranslateX(0); return; } //
  fue un clic
  const threshold = velocity > 0.5 ? 50 : 100;
  if (translateX > threshold && currentIndex > 0) prevSlide();
  else if (translateX < -threshold && currentIndex < maxIndex) nextSlide();
  setTranslateX(0);
};

const handleMouseLeave = () => { if (isDragging) handleMouseUp(); };

```

Conéctalos a la capa del carrusel junto con el ref. El cursor va en `style` porque depende de `isDragging`:

3.7 – capa del carrusel

```

<div
  ref={carouselRef}
  role="region"
  aria-roledescription="carrusel"
  aria-label="Soluciones"
  onMouseDown={handleMouseDown}
  onMouseMove={handleMouseMove}
  onMouseUp={handleMouseUp}
  onMouseLeave={handleMouseLeave}
  className="w-full select-none"
  style={{ cursor: isDragging ? "grabbing" : "grab", touchAction: "pan-y" }}
>

```

`touchAction: "pan-y"` le dice al navegador que en touch el gesto vertical es suyo (scroll de la página) y el horizontal lo maneja tu código.

3.8 Touch JS

Swipe táctil

Se registra como listener **nativo** con `{ passive: false }`. React marca `onTouchMove` como pasivo, y en un listener pasivo `preventDefault()` no frena el scroll: al swipear en horizontal

se movería toda la página junto con el carrusel. Con el listener nativo, desde los 10px de movimiento horizontal la página se queda quieta.

3.8

```
// — Swipe táctil — listener nativo para poder frenar el scroll de la página —
useEffect(() => {
  const el = carouselRef.current;
  if (!el) return;

  let localStartX = 0;
  let localStartTime = 0;
  let localTranslateX = 0;

  const onTouchStart = (e) => {
    localStartX = e.touches[0].clientX;
    localStartTime = Date.now();
    localTranslateX = 0;
    setTranslateX(0);
  };

  const onTouchMove = (e) => {
    const diff = e.touches[0].clientX - localStartX;
    localTranslateX = Math.max(-200, Math.min(200, diff));
    if (Math.abs(diff) > 10) e.preventDefault();
    setTranslateX(localTranslateX);
  };

  const onTouchEnd = () => {
    const elapsed = Date.now() - localStartTime;
    const velocity = Math.abs(localTranslateX) / elapsed;
    const threshold = velocity > 0.5 ? 30 : 50;
    // Lee los refs: este efecto corre una sola vez, así que
    // currentIndex/maxIndex quedarían fijos al primer render
    if (localTranslateX > threshold && currentIndexRef.current > 0) {
      setCurrentIndex((p) => p - 1);
    } else if (localTranslateX < -threshold && currentIndexRef.current <
maxIndexRef.current) {
      setCurrentIndex((p) => p + 1);
    }
    setTranslateX(0);
  };

  el.addEventListener("touchstart", onTouchStart, { passive: true });
  el.addEventListener("touchmove", onTouchMove, { passive: false });
  el.addEventListener("touchend", onTouchEnd, { passive: true });
  return () => {
    el.removeEventListener("touchstart", onTouchStart);
  };
});
```

```

    el.removeEventListener("touchmove", onTouchMove);
    el.removeEventListener("touchend", onTouchEnd);
  };
}, []);

```

Con el dedo los umbrales son más bajos (30 o 50px) que con mouse (50 o 100px), porque los gestos son más cortos.

3.9 Botón JS

Botón “Ver proyectos”

Igual que en `BentoCard`: la card recibe un `onClick` por props y la sección decide qué hacer. Son tres cambios.

3.9.1 – en `SolutionSection`

```

// — Botón "Ver proyectos" —
const handleCardClick = (solutionId, isActive) => {
  if (isDragging || Math.abs(translateX) > 5) return; // venía de un arrastre
  if (isActive) navigate(`/soluciones/${solutionId}`);
};

```

3.9.2 – en el track

```

<SolutionCard solution={solution} onClick={handleCardClick} />

```

3.9.3 – en `SolutionCard`

```

function SolutionCard({ solution, onClick }) {
  // ...
  {solution.active ? (
    <button
      type="button"
      onClick={(e) => { e.stopPropagation();
onClick(solution.id, solution.active); }}
      className={` ${BTN_BASE} ${BTN_ENABLED}`}
    >
      Ver proyectos
      <ArrowUpRight className="h-3 w-3" />
    </button>
  ) : (
    <button type="button" disabled title="Próximamente"

```

```

className={` ${BTN_BASE} ${BTN_DISABLED}`}>
      <Clock className="h-3 w-3" />
      Ver proyectos
    </button>
  )}
// ...
}

```

El botón deshabilitado no necesita `onClick`, porque un botón `disabled` no dispara clics. `stopPropagation()` evita que el clic suba hasta el carrusel.

Antes de probar el botón: 1) Las 4 soluciones de `solutions.js` tienen `active: false`; pon `true` en al menos una. 2) `AppRoutes.jsx` solo tiene la ruta `/`. Hasta que crees `<Route path="/soluciones/:id" ... />`, el botón cambia la URL pero se ve una página vacía.

Archivo final

Las tres partes juntas. Úsalo para comparar con lo que construiste, no para saltarte los pasos.

```

src/components/Home/SoluctionSection.jsx

import { useState, useEffect, useRef } from "react";
import { useNavigate } from "react-router-dom";

//datos de las cards
import { solutions } from '../../data/solutions';

//iconos
import { ArrowUpRight, Clock } from 'lucide-react'

// ===== ESTILOS =====

//Card base - `relative` para el fondo de puntos.
//Alto: la sección nunca pasa del 95% de la ventana. Se resta lo fijo
//(pt + dots + pb: 23.5rem en móvil, 26rem desde md) y se limita entre 260px y 330px
const CARD_BASE =
  'relative flex h-[clamp(260px,calc(95svh_-_23.5rem),330px)] md:h-[clamp(260px,calc(95svh_-_26rem),330px)] flex-col overflow-hidden rounded-[20px] border border-black/5 bg-gray-100 ' +
  'shadow-[0_2px_12px_0_rgba(0,0,0,0.08),0_1px_3px_0_rgba(0,0,0,0.05)] dark:border-white/10 dark:bg-zinc-800'

```

```

//Hover solo en cards activas (sin cursor-pointer: la acción es el botón)
const CARD_ACTIVE =
    'transition duration-500 hover:-translate-y-1 ' +
    'hover:shadow-[0_12px_32px_rgba(0,0,0,0.13),0_2px_8px_rgba(0,0,0,0.08)]'

//Fondo de puntos – reemplaza a DotPatternCanvas con CSS puro
const DOT_PATTERN = {
    backgroundImage: 'radial-gradient(circle, rgba(128,128,128,0.13) 1.1px,
transparent 1.2px)',
    backgroundSize: '18px 18px',
    maskImage: 'linear-gradient(to bottom, black, transparent)',
    WebkitMaskImage: 'linear-gradient(to bottom, black, transparent)',
}

//Botón "Ver proyectos" – mismos estilos que los CTAs de MyServicesSection
const BTN_BASE =
    'inline-flex w-full items-center justify-center gap-1.5 rounded-lg px-3 py-2
text-[11.5px] font-semibold'

const BTN_ENABLED =
    'bg-zinc-800 text-white hover:bg-blue-600 transition-colors duration-200 ' +
    'dark:bg-zinc-100 dark:text-zinc-900 dark:hover:bg-blue-500 dark:hover:text-
white'

const BTN_DISABLED =
    'cursor-not-allowed bg-zinc-200/70 text-zinc-400 dark:bg-zinc-700/50 dark:text-
zinc-500'

//Ancho de cada slide – 1 / 2 / 3 cards (gap-4 = 1rem entre cards)
const SLIDE_WIDTH = 'w-full md:w-[calc((100%_-_1rem)/2)] lg:w-[calc((100%_-_
2rem)/3)]'

//Dots – diseño original de MyServicesSection
const DOT_BASE = 'h-2 rounded-full transition-all duration-300'
const DOT_ACTIVE = 'w-8 bg-zinc-600 dark:bg-zinc-300'
const DOT_INACTIVE = 'w-2 bg-zinc-300 hover:bg-zinc-400 dark:bg-zinc-600
dark:hover:bg-zinc-500'

const GAP = 16; // px – igual a gap-4 del track

// ===== CARD =====
function SolutionCard({ solution, onClick }) {
    return (
        <article className={` ${CARD_BASE} ${solution.active ? CARD_ACTIVE : ''}`>
            { /* Fondo de puntos */ }
            <div aria-hidden="true" className="pointer-events-none absolute inset-0"
style={DOT_PATTERN} />

            { /* Área visual */ }
        </article>
    )
}

```

```

<figure className="relative z-[1] min-h-20 flex-1 overflow-hidden">
  {solution.image ? (
    <img src={solution.image} alt={solution.alt} loading="lazy"
    draggable={false} className="pointer-events-none h-full w-full object-cover" />
  ) : (
    <div className="flex h-full w-full flex-col items-center justify-
    center gap-3 bg-zinc-400/20 dark:bg-zinc-950/40">
      <svg width="28" height="28" viewBox="0 0 28 28" fill="none"
      aria-hidden="true" className="text-zinc-500 opacity-30 dark:text-zinc-400">
        <path d="M14 3v10m0 0 4-4m-4 4-4-4" stroke="currentColor"
        strokeWidth="1.6" strokeLinecap="round" strokeLinejoin="round" />
        <path d="M4 18v4a2 2 0 0 0 2 2h16a2 2 0 0 0 2 0 0 0 2-2v-4"
        stroke="currentColor" strokeWidth="1.6" strokeLinecap="round" />
      </svg>
      <span className="text-[10px] uppercase tracking-[0.5px] text-
      zinc-400 dark:text-zinc-500">
        Proyectos en camino
      </span>
    </div>
  )}
  <div className="pointer-events-none absolute inset-x-0 bottom-0 h-16
  bg-linear-to-b from-transparent to-gray-100 dark:to-zinc-800" />
</figure>

{/* Contenido textual */}
<div className="relative z-[2] flex shrink-0 flex-col px-5 pt-3 pb-4">
  <h3 className="mb-2 text-[clamp(1.05rem,1.6vw,1.25rem)] leading-tight
  font-bold tracking-tight text-zinc-800 dark:text-zinc-50">
    {solution.title}
  </h3>
  <hr className="mb-3 border-zinc-200 dark:border-white/10" />
  <p className="line-clamp-3 text-[12.5px] leading-[1.65] text-zinc-500
  dark:text-zinc-400">
    {solution.description}
  </p>

  {/* CTA - habilitado solo si la solución está activa */}
  <div className="mt-auto pt-3">
    {solution.active ? (
      <button
        type="button"
        onClick={(e) => { e.stopPropagation();
onClick(solution.id, solution.active); }}
        className={` ${BTN_BASE} ${BTN_ENABLED}`} `
      >
        Ver proyectos
      <ArrowUpRight className="h-3 w-3" />
    </button>
    ) : (

```

```

        <button type="button" disabled title="Próximamente"
className={` ${BTN_BASE} ${BTN_DISABLED}`}>
            <Clock className="h-3 w-3" />
            Ver proyectos
        </button>
    )}
</div>
</div>
</article>
)
}

```

```

// ===== SECCIÓN =====
const SolutionSection = () => {
    const navigate = useNavigate();

    // — Estado del carrusel —
    const [currentIndex, setCurrentIndex] = useState(0);
    const [slidesPerView, setSlidesPerView] = useState(3);
    const [isDragging, setIsDragging] = useState(false);
    const [startX, setStartX] = useState(0);
    const [translateX, setTranslateX] = useState(0);
    const [startTime, setStartTime] = useState(0);

    // — Refs del carrusel (solo los necesarios) —
    const carouselRef = useRef(null);
    const isDraggingRef = useRef(false);
    const currentIndexRef = useRef(0);
    const maxIndexRef = useRef(0);

    const maxIndex = Math.max(0, solutions.length - slidesPerView);
    currentIndexRef.current = currentIndex;
    maxIndexRef.current = maxIndex;

    // — Cards por vista según el ancho de la ventana —
    useEffect(() => {
        const handleResize = () => {
            if (window.innerWidth ≥ 1024) setSlidesPerView(3);
            else if (window.innerWidth ≥ 768) setSlidesPerView(2);
            else setSlidesPerView(1);
        };
        handleResize();
        window.addEventListener("resize", handleResize);
        return () => window.removeEventListener("resize", handleResize);
    }, []);

    // — Corrige el índice si queda fuera de rango —
    useEffect(() => {
        const maxPossible = Math.max(0, solutions.length - slidesPerView);

```

```

    if (currentIndex > maxPossible) setCurrentIndex(maxPossible);
  }, [slidesPerView, currentIndex]);

const nextSlide = () => setCurrentIndex(p => p ≥ maxIndex ? 0 : p + 1);
const prevSlide = () => setCurrentIndex(p => p ≤ 0 ? maxIndex : p - 1);

// — Arrastre con mouse —
const handleMouseDown = (e) => {
  if (e.target.tagName === "BUTTON" || e.target.closest("button")) return;
  setIsDragging(true);
  isDraggingRef.current = true;
  setStartX(e.pageX);
  setStartTime(Date.now());
  setTranslateX(0);
  e.preventDefault();
};

const handleMouseMove = (e) => {
  if (!isDraggingRef.current) return;
  e.preventDefault();
  setTranslateX(Math.max(-200, Math.min(200, e.pageX - startX)));
};

const handleMouseUp = () => {
  if (!isDraggingRef.current) return;
  setIsDragging(false);
  isDraggingRef.current = false;
  const elapsed = Date.now() - startTime;
  const velocity = Math.abs(translateX) / elapsed;
  if (Math.abs(translateX) < 5 && elapsed < 200) { setTranslateX(0); return; }
  const threshold = velocity > 0.5 ? 50 : 100;
  if (translateX > threshold && currentIndex > 0) prevSlide();
  else if (translateX < -threshold && currentIndex < maxIndex) nextSlide();
  setTranslateX(0);
};

const handleMouseLeave = () => { if (isDragging) handleMouseUp(); };

// — Swipe táctil — listener nativo para poder frenar el scroll de la página —
useEffect(() => {
  const el = carouselRef.current;
  if (!el) return;

  let localStartX = 0;
  let localStartTime = 0;
  let localTranslateX = 0;

  const onTouchStart = (e) => {
    localStartX = e.touches[0].clientX;

```

```

    localStartTime = Date.now();
    localTranslateX = 0;
    setTranslateX(0);
  };

  const onTouchMove = (e) => {
    const diff = e.touches[0].clientX - localStartX;
    localTranslateX = Math.max(-200, Math.min(200, diff));
    if (Math.abs(diff) > 10) e.preventDefault();
    setTranslateX(localTranslateX);
  };

  const onTouchEnd = () => {
    const elapsed = Date.now() - localStartTime;
    const velocity = Math.abs(localTranslateX) / elapsed;
    const threshold = velocity > 0.5 ? 30 : 50;
    if (localTranslateX > threshold && currentIndexRef.current > 0) {
      setCurrentIndex((p) => p - 1);
    } else if (localTranslateX < -threshold && currentIndexRef.current <
maxIndexRef.current) {
      setCurrentIndex((p) => p + 1);
    }
    setTranslateX(0);
  };

  el.addEventListener("touchstart", onTouchStart, { passive: true });
  el.addEventListener("touchmove", onTouchMove, { passive: false });
  el.addEventListener("touchend", onTouchEnd, { passive: true });
  return () => {
    el.removeEventListener("touchstart", onTouchStart);
    el.removeEventListener("touchmove", onTouchMove);
    el.removeEventListener("touchend", onTouchEnd);
  };
}, []);

// — Desplazamiento del track —
const getTransformValue = () => {
  if (!carouselRef.current) return "translateX(0px)";
  const containerWidth = carouselRef.current.offsetWidth;
  const cardWidth = (containerWidth - (slidesPerView - 1) * GAP) /
slidesPerView;
  const offset = currentIndex * (cardWidth + GAP);
  return `translateX(${ -offset + translateX }px)`;
};

// — Botón "Ver proyectos" —
const handleCardClick = (solutionId, isActive) => {
  if (isDragging || Math.abs(translateX) > 5) return;
  if (isActive) navigate(`/soluciones/${solutionId}`);

```



```

        <g opacity="0.4">
            <rect width="44" height="39" x="92" y="38"
fill="currentColor" className="text-zinc-400/20 dark:text-zinc-600/20" />
            <rect width="44" height="39" x="182" y="114"
fill="currentColor" className="text-zinc-400/20 dark:text-zinc-600/20" />
            <rect width="44" height="39" x="135" y="190"
fill="currentColor" className="text-zinc-400/20 dark:text-zinc-600/20" />
        </g>
    </svg>
</div>

{/* Contenedor dots + carrusel */}
<div className="relative z-10 px-6 pt-70 md:pt-80 pb-12 sm:px-8 md:px-
10">

    {/* Dots - uno por posición posible */}
    {maxIndex > 0 && (
        <div className="mt-4 mb-6 flex items-center gap-2">
            {Array.from({ length: maxIndex + 1 }).map((_, i) => (
                <button
                    key={i}
                    type="button"
                    onClick={() => setCurrentIndex(i)}
                    aria-label={`Ir al slide ${i + 1}`}
                    className={` ${DOT_BASE} ${i === currentIndex ?
DOT_ACTIVE : DOT_INACTIVE}`}
                />
            ))}
        </div>
    )}

    {/* Wrapper - nunca se transforma */}
    <div className="relative w-full">
        {/* Clip - recorta a los lados sin cortar sombras */}
        <div style={{ clipPath: "inset(-16px -12px -16px -12px)",
padding: "16px 12px", margin: "-16px -12px" }}>
            {/* Carrusel - se mide y recibe el arrastre */}
            <div
                ref={carouselRef}
                role="region"
                aria-roledescription="carrusel"
                aria-label="Soluciones"
                onMouseDown={handleMouseDown}
                onMouseMove={handleMouseMove}
                onMouseUp={handleMouseUp}
                onMouseLeave={handleMouseLeave}
                className="w-full select-none"
                style={{ cursor: isDragging ? "grabbing" : "grab",
touchAction: "pan-y" }}

```

```

    >
    { /* Track */
    <ul
      className="flex gap-4"
      style={{
        transform: getTransformValue(),
        transition: isDragging ? "none" : "transform
300ms ease-out",
      }}
    >
      {solutions.map((solution) => (
        <li key={solution.id} className={`shrink-0
${SLIDE_WIDTH}`}>
          <SolutionCard solution={solution} onClick=
{handleCardClick} />
        </li>
      ))}
    </ul>
  </div>
</div>
</div>
</div>
</section>
)
}

export default SolutionSection

```

A tener en cuenta (heredado de MyServicesSection, no impide que funcione):

- **Redimensionar sin cruzar un breakpoint.** `getTransformValue()` mide durante el render. Si cambias el ancho de 1100 a 1300px, React no vuelve a renderizar y el track queda corrido hasta la siguiente interacción. Solo se nota si el carrusel no está en la primera card.
- **Avisos de ESLint.** Tu proyecto usa `eslint-plugin-react-hooks` v7. Es posible que marque la escritura de `currentIndexRef.current` durante el render y el `setCurrentIndex` dentro del efecto del paso 3.3. En ejecución funciona igual.

Pruebas

Al terminar cada parte, revisa en DevTools con el modo dispositivo (Ctrl+Shift+M).

- ☐ **Parte 1:** con un `min-h` temporal, el panel se ve a la izquierda, con el título, la línea, el brillo en la esquina y la cuadrícula inclinada. También en modo oscuro.
- ☐ **Parte 2:** 3 cards desde 1024px, 2 entre 768 y 1023px y 1 por debajo. La cuarta no asoma.
- ☐ **Parte 2:** en móvil no hay scroll interno en la sección, y la sección completa no pasa del 95% del alto de la ventana (en ventanas de más de unos 670px de alto en móvil y 712px en escritorio).
- ☐ **Parte 2:** los dots se ven como en el original: el primero largo y oscuro, el segundo corto y claro.
- ☐ **Parte 3:** 2 dots a 1280px, 3 a 800px y 4 a 400px. Cada dot mueve el carrusel exactamente una card.
- ☐ **Parte 3:** al arrastrar con el mouse avanza o retrocede una card y se detiene en los extremos. Si sacas el mouse mientras arrastras, el track no queda “pegado”.
- ☐ **Parte 3:** en touch, un swipe horizontal mueve el carrusel sin mover la página, y uno vertical hace scroll normal.
- ☐ **Parte 3:** ve a la última card en móvil y agranda la ventana: el carrusel queda en la última posición válida, sin espacio vacío.
- ☐ **Parte 3:** con una solución en `active: true`, su botón cambia la URL a `/soluciones/<id>` y no arranca un arrastre. Los botones deshabilitados no hacen nada ni muestran errores.

Tutorial para MyPortfolio · SoluctionSection · diseño + funcionalidad. Las animaciones con GSAP van en la siguiente fase.